

explain me random, json modules

Absolutely. Since you're learning Python for **data engineering and Spark Structured Streaming**, `random` and `json` are especially useful. `random` helps you generate fake/test data, while `json` helps you create and read the JSON data that you can feed into Spark.

1. Python `random` Module

The `random` module is used to generate **random values**.

</> Python



▶ Run

```
import random
```

You don't need to install it because it is part of Python's standard library.

1.1 `random.random()`

Generates a random floating-point number between **0.0 and 1.0**.

</> Python



▶ Run

```
import random

x = random.random()

print(x)
```

Example:

0.7348291



Every execution can produce a different value.

Range

$0.0 \leq x < 1.0$



1.2 random.randint()

Generates a random integer between two values, **including both ends**.

</> Python



▶ Run

```
import random

x = random.randint(1, 10)

print(x)
```

Possible results:

```
1
2
3
...
10
```



For example, generate a random age:

</> Python



▶ Run

```
age = random.randint(18, 60)

print(age)
```

1.3 random.randrange()

Similar to Python's `range()` .

</> Python



▶ Run

```
random.randrange(start, stop)
```

The `stop` value is **not included**.

</> Python



▶ Run

```
x = random.randrange(1, 10)

print(x)
```

Possible:

1 → 9



You can also specify a step:

</> Python



Run

```
x = random.randrange(0, 100, 10)

print(x)
```

Possible:

```
0
10
20
30
...
90
```



1.4 random.uniform()

Generates a random floating-point number between two values.

</> Python



Run

```
price = random.uniform(10, 100)

print(price)
```

Example:

57.38291



This is useful when creating fake **prices**.

For example:

</> Python



Run

```
price = round(random.uniform(10, 100), 2)

print(price)
```

Output:

57.38



1.5 random.choice()

Selects one random element from a list.

</> Python



▶ Run

```
import random

cities = ["Cairo", "Giza", "Alexandria", "Luxor"]

city = random.choice(cities)

print(city)
```

Possible:

Giza



Another execution:

Cairo



This is extremely useful for generating fake data.

</> Python



▶ Run

```
products = ["Laptop", "Phone", "Tablet", "Monitor"]

product = random.choice(products)
```

1.6 random.choices()

Selects **multiple elements**.

</> Python



▶ Run

```
colors = ["red", "blue", "green"]

result = random.choices(colors, k=5)

print(result)
```

Example:

```
['blue', 'green', 'blue', 'red', 'blue']
```



Important difference:

```
</> Python

random.choice()
```



▶ Run

→ one item

```
</> Python

random.choices()
```



▶ Run

→ multiple items, and items can repeat.

1.7 random.sample()

Selects multiple **unique** elements.

```
</> Python

numbers = [1, 2, 3, 4, 5]

result = random.sample(numbers, 3)

print(result)
```



▶ Run

Example:

```
[5, 2, 4]
```



No duplicate values.

Compare:

</> Python



▶ Run

```
random.choices([1, 2, 3, 4, 5], k=3)
```

could produce:

[2, 2, 5]



while:

</> Python



▶ Run

```
random.sample([1, 2, 3, 4, 5], 3)
```

cannot produce duplicates.

1.8 random.shuffle()

Randomly rearranges a list.

</> Python



▶ Run

```
numbers = [1, 2, 3, 4, 5]
```

```
random.shuffle(numbers)
```

```
print(numbers)
```

Possible:

[3, 5, 1, 4, 2]



Important:

`shuffle()` changes the original list.

1.9 Random Boolean

You can generate a random `True` or `False` .

</> Python



▶ Run

```
is_active = random.choice([True, False])

print(is_active)
```

Useful for generating fake records:

```
</> Python

users = [
    {"id": 1, "active": True},
    {"id": 2, "active": False}
]
```



▶ Run

1.10 random.seed()

Normally, random values change every time.

```
</> Python

import random

print(random.randint(1, 100))
```



▶ Run

But sometimes you want **repeatable random data**.

Use:

```
</> Python

random.seed(42)
```



▶ Run

Example:

```
</> Python

import random

random.seed(42)

print(random.randint(1, 100))
print(random.randint(1, 100))
print(random.randint(1, 100))
```



▶ Run

Running the same program again with the same seed gives the same sequence.

This is useful when testing your data pipeline.

2. Python json Module

The `json` module is used to work with **JSON data**.

```
</> Python
```



▶ Run

```
import json
```

JSON is extremely important in data engineering.

For example:

```
</> JSON
```



```
{
  "id": 1,
  "name": "Youssef",
  "age": 24
}
```

Python dictionary:

```
</> Python
```



▶ Run

```
{
  "id": 1,
  "name": "Youssef",
  "age": 24
}
```

They look similar, but they are not the same thing.

2.1 Python Dictionary → JSON String

Use:

```
</> Python
```



▶ Run

```
json.dumps()
```

Example:

</> Python



Run

```
import json

data = {
    "id": 1,
    "name": "Youssef",
    "age": 24
}

json_data = json.dumps(data)

print(json_data)
```

Output:

</> JSON



```
{"id": 1, "name": "Youssef", "age": 24}
```

The result is a **string**.

Check:

</> Python



Run

```
print(type(json_data))
```

Output:

<class 'str'>



2.2 Pretty JSON

You can make JSON easier to read:

</> Python



Run

```
json_data = json.dumps(data, indent=4)

print(json_data)
```

Output:

</> JSON



```
{
  "id": 1,
  "name": "Youssef",
  "age": 24
}
```

2.3 Python Dictionary → JSON File

This is very important for your Spark streaming practice.

Suppose:

</> Python



▶ Run

```
data = {
  "id": 1,
  "name": "Youssef",
  "age": 24
}
```

Save it to a JSON file:

</> Python



▶ Run

```
import json

with open("user.json", "w") as file:
    json.dump(data, file, indent=4)
```

Now you have:

user.json



Containing:

</> JSON



```
{
  "id": 1,
  "name": "Youssef",
  "age": 24
}
```

Notice:

```
</> Python  
  
json.dump()
```



▶ Run

writes directly to a **file**.

2.4 JSON File → Python Dictionary

Use:

```
</> Python  
  
json.load()
```



▶ Run

Example:

```
</> Python  
  
import json  
  
with open("user.json", "r") as file:  
    data = json.load(file)  
  
print(data)
```



▶ Run

Output:

```
</> Python  
  
{'id': 1, 'name': 'Youssef', 'age': 24}
```



▶ Run

Now:

```
</> Python  
  
print(type(data))
```



▶ Run

Output:

```
<class 'dict'>
```



2.5 dump() vs dumps()

This is very important.

`json.dump()`

Python object → **JSON file**

</> Python

```
json.dump(data, file)
```



▶ Run

`json.dumps()`

Python object → **JSON string**

</> Python

```
json_string = json.dumps(data)
```



▶ Run

Think:

```
dump → file  
dumps → string
```



2.6 load() vs loads()

Same idea.

`json.load()`

JSON file → Python object

</> Python

```
data = json.load(file)
```



▶ Run

`json.loads()`

JSON string → Python object

</> Python

```
data = json.loads(json_string)
```



▶ Run

Think:

load → file

loads → string



3. JSON Data Types

JSON supports:

JSON	Python
object	dict
array	list
string	str
number	int / float
true	True
false	False
null	None

Example JSON:

</> JSON

```
{
  "id": 10,
  "name": "Laptop",
  "price": 1200.50,
  "available": true,
  "tags": ["computer", "electronics"],
  "discount": null
}
```



Python equivalent:

</> Python



▶ Run

```
{
  "id": 10,
  "name": "Laptop",
  "price": 1200.50,
  "available": True,
  "tags": ["computer", "electronics"],
  "discount": None
}
```

Notice:

JSON:

</> JSON



```
true
```

Python:

</> Python



▶ Run

```
True
```

4. Combining random + json

This is the **important part for you**.

You can use `random` to generate fake data and `json` to save it.

For example:

</> Python



▶ Run

```
import random
import json

data = {
    "id": random.randint(1, 1000),
    "product": random.choice(["Laptop", "Phone", "Tablet"]),
    "price": round(random.uniform(100, 2000), 2),
    "quantity": random.randint(1, 10)
}

print(data)
```

Possible output:

</> Python



Run

```
{
  'id': 582,
  'product': 'Laptop',
  'price': 1345.72,
  'quantity': 3
}
```

Save it:

</> Python



Run

```
with open("event.json", "w") as file:
    json.dump(data, file, indent=4)
```

The file:

</> JSON



```
{
  "id": 582,
  "product": "Laptop",
  "price": 1345.72,
  "quantity": 3
}
```

5. Generate Many JSON Records

This is even more useful for Spark.

</> Python



Run

```
import random
import json

products = ["Laptop", "Phone", "Tablet", "Monitor"]

records = []

for i in range(10):

    record = {
        "id": i + 1,
        "product": random.choice(products),
        "price": round(random.uniform(100, 2000), 2),
        "quantity": random.randint(1, 10)
    }

    records.append(record)

with open("events.json", "w") as file:
    json.dump(records, file, indent=4)
```

The file will contain an array:

```
</> JSON

[
  {
    "id": 1,
    "product": "Phone",
    "price": 500.25,
    "quantity": 2
  },
  {
    "id": 2,
    "product": "Laptop",
    "price": 1450.75,
    "quantity": 1
  }
]
```

6. JSON Lines — Very Important for Streaming

For **Spark Structured Streaming**, you will often work with **JSON Lines** / **NDJSON**.

Instead of:

</> JSON



```
[
  {"id": 1, "product": "Phone"},
  {"id": 2, "product": "Laptop"},
  {"id": 3, "product": "Tablet"}
]
```

you can have:

</> JSON



```
{"id": 1, "product": "Phone"}
{"id": 2, "product": "Laptop"}
{"id": 3, "product": "Tablet"}
```

Each line is a separate JSON object.

This format is very convenient for streaming because you can continuously add new records.

Python:

</> Python



Run

```
import json

records = [
    {"id": 1, "product": "Phone"},
    {"id": 2, "product": "Laptop"},
    {"id": 3, "product": "Tablet"}
]

with open("events.json", "w") as file:

    for record in records:
        file.write(json.dumps(record) + "\n")
```

Result:

events.json



```
{"id": 1, "product": "Phone"}
{"id": 2, "product": "Laptop"}
{"id": 3, "product": "Tablet"}
```

This is the style you should become comfortable with for **file-based Spark Structured**

Streaming.

7. Generate Streaming Data Continuously

You can combine this with `time.sleep()` later to simulate a real streaming source:

Python



Run

```
import json
import random
import time

products = ["Laptop", "Phone", "Tablet", "Monitor"]

while True:

    event = {
        "id": random.randint(1, 100000),
        "product": random.choice(products),
        "price": round(random.uniform(100, 2000), 2),
        "quantity": random.randint(1, 10)
    }

    with open("events.json", "a") as file:
        file.write(json.dumps(event) + "\n")

    print(event)

    time.sleep(2)
```

Every 2 seconds, a new JSON record is generated.

Conceptually:

```
Python
|
| generates random event
▼
JSON file
|
| new records
▼
Spark Structured Streaming
|
▼
Transformations
|
▼
HDFS / Parquet / Delta
```



This is a very useful mini-lab for learning streaming **without Kafka**.

The functions you should memorize

random

</> Python



▶ Run

```
random.random()
random.randint()
random.randrange()
random.uniform()
random.choice()
random.choices()
random.sample()
random.shuffle()
random.seed()
```

json

</> Python

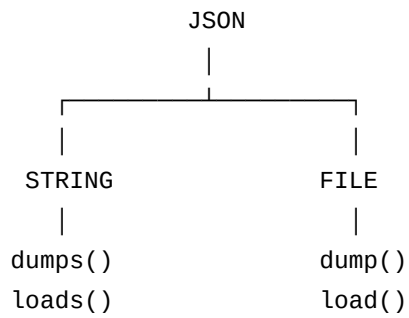


▶ Run

```
json.dumps()    # Python → JSON string
json.loads()    # JSON string → Python

json.dump()     # Python → JSON file
json.load()     # JSON file → Python
```

The most important mental model is:



And for your Spark practice:

```
random
↓
generate fake event
↓
json.dumps()
↓
JSON Lines file
↓
Spark readStream()
↓
transform
↓
writeStream()
↓
Parquet / Delta / HDFS
```



ChatGPT can make mistakes. Check important info.